

অপারেটিং সিস্টেমের ভূমিকা

(Introduction to Operating Systems)

প্রফেসর চেস্টার রেবেরো

(Prof. Chester Rebeiro)

কম্পিউটার বিজ্ঞান ও প্রকৌশল বিভাগ

(Department of Computer Science and Engineering),

ইন্ডিয়ান ইনস্টিটিউট অফ টেকনোলজি, মাদ্রাজ

(Indian Institute of Technology, Madras)

সপ্তাহ – 06

বক্তৃতা – 26

সিঙ্ক্রোনাইজেশনের জন্য বেকারি অ্যালগরিদম

(The Bakery Algorithm for Synchronization)

এই ভিডিওটিতে আমরা বেকারি অ্যালগরিদম দেখতে পাব, যা Critical section এর একটি সফটওয়্যার সমাধান। মূলত পিটারসন এর সমাধানের সাথে তুলনা করা হয় যা আমরা আগে দেখেছি, এই অ্যালগরিদম বেশি সংখ্যার প্রসেসের জন্য সবচেয়ে উপযুক্ত। মূলত যখন আমাদের প্রসেস সংখ্যা 2 এর চেয়ে বড়, তখন বেকারি অ্যালগরিদম দক্ষতার সাথে কাজ করবে।

(স্লাইড সময় : 00:43 দেখুন)

সুতরাং, বেকারি অ্যালগরিদমটি লেসলি ল্যামপটের দ্বারা উদ্ভাবিত হয়েছিল এবং এখানে আপনি এই বিশেষ ওয়েবসাইট থেকে আরও তথ্য পেতে পারেন। সুতরাং, বেকারি অ্যালগরিদমের অপরিহার্য দৃষ্টিভঙ্গি হচ্ছে বেকারী ও ব্যাংকগুলি থেকে অনুপ্রেরণা। কিছু বেকারিতে, আমরা দেখি যে আমরা যখন বেকারিতে প্রবেশ করি, তখন একটি একটি বিশেষ token দেওয়া হয়, এখানে আমাদের একটি সংখ্যা দিয়ে একটি token দেওয়া হয়। সুতরাং এই বিশেষ ক্ষেত্রে, আমাদের একটি টোকেন সংখ্যা আছে 196। এখন পর্যন্ত কিছুক্ষণ অপেক্ষা করতে হবে যখন টোকেন নান্দ্রার 196 কে ডাকা হয়। সুতরাং, এখানে একটি প্রদর্শন আছে যা পর্যায়ক্রমে একটি টোকেন সংখ্যা নির্ধারণ করবে এবং যখন token সংখ্যা 196 প্রদর্শিত হবে তখন আপনি বেকারি থেকে আপনার খাবার পেতে পারবেন

এবং আপনি খেতে পারবেন। সুতরাং, মূলত যখন আমরা synchronization এর দৃষ্টিভঙ্গি থেকে এটি দেখি, তখন আমরা দেখতে পাই যে আমরা একটি নির্দিষ্ট counter ব্যবহারকে synchronize করার চেষ্টা করছি। সুতরাং যাদের এই ধরনের একটি token আছে যাতে তাদের সংখ্যা বলা হয় ততক্ষণ অপেক্ষা করা উচিত, তারপর ক্রমানুসারে সংখ্যা বলা হলে প্রতিটি ব্যক্তি counter এ যায়, এবং তিনি যাই হোক না কেন সংগ্রহ করতে পারবেন আর খেতে পারবেন।

(স্লাইড সময় : 02:16 দেখুন)

সুতরাং, আমরা দেখব কিভাবে বেকারি অ্যালগরিদমটি critical section সমস্যা সমাধান করতে ব্যবহৃত হয়। সুতরাং, আমরা বেকারি অ্যালগরিদম সরলীকৃত বিশ্লেষণ দিয়ে শুরু করব। সুতরাং, এই বিশেষ অ্যালগরিদমটি critical section সমস্যা সমাধানের জন্য ব্যবহার করা হয় যখন N প্রসেস জড়িত থাকে এবং এই সমস্ত N প্রসেসগুলি একই critical section এ প্রবেশ করতে চায়। এখন এমন একটি global data রয়েছে যা এই N প্রসেসগুলির মধ্যে share করা হয় এবং এই data num হিসাবে পরিচিত। সুতরাং, num এর দৈর্ঘ্য N এর আকারের সমান; মূলত প্রতিটি প্রক্রিয়ায় এই num array তে একটি নির্দিষ্ট index রয়েছে। সুতরাং, উদাহরণস্বরূপ প্রক্রিয়া 0 এর num 0 এর অনুরূপ একটি flag থাকবে, প্রসেস 1 এর num 1, প্রসেস 2 এর num 2 ইত্যাদি। দ্বিতীয়ত, execution এর শুরুতে, num এর মানটি 0 নির্ধারিত হয়। এখন একটি critical section এ প্রবেশ করার জন্য, একটি প্রক্রিয়া প্রথমে i এর মানের সঙ্গে lock চালু করতে হবে। সুতরাং, উদাহরণস্বরূপ i এখানে প্রক্রিয়া নম্বর, একটি প্রক্রিয়া id হয়। সুতরাং, আমাদের N প্রসেস আছে তাই, i এর মান 0 থেকে N-1 পর্যন্ত হতে পারে। এবং critical section এ প্রস্থানের সময়ে, function unlock i invoke করা হয় যেখানে num i 0 তে সেট করা হয়। তাই আসুন দেখুন কি লক এবং আনলক আসলে অভ্যন্তরীণভাবে করছেন। সুতরাং, যখন একটি প্রক্রিয়া i লক আমন্ত্রণ জানায় যেখানে i তার সংখ্যা, তার num ভ্যালু; সুতরাং, i এর সর্বোচ্চ মান সেট করা হয়, মূলত এই বিশেষ function MAX সমস্ত সংখ্যা সংশ্লিষ্ট সমস্ত প্রসেসের এবং যা থেকে সর্বোচ্চ সংখ্যা পেতে এক যোগ কর হয়। সুতরাং, num i সর্বোচ্চ সংখ্যক নম্বর পেতে যাচ্ছে যা shared num array এর মধ্যে উপস্থিত। দ্বিতীয় একটা for লুপ আছে, যা সমস্ত প্রসেসগুলির মধ্যে scan করে। সুতরাং, p = 0 থেকে p < N ++ p । এবং এটির মধ্যে একটি for লুপ রয়েছে আর ওখানে while লুপ যা দুটি জিনিস পরীক্ষা করে, এটি পরীক্ষা করে যে num p এর সংখ্যা 0 এর সমান নয় এবং num p এর সংখ্যা i এর চেয়ে কম হয়। সুতরাং, মূলত এই while লুপ ভেঙ্গে যাবে, যখন হয় p এর সংখ্যা 0 হয় বা এই বিশেষ শর্ত false হয়। সুতরাং, মূলত i num p এর কম বা সমান হয়। সুতরাং, মূলত, i প্রক্রিয়াটি critical section এ প্রবেশ করবে, শুধুমাত্র যদি এটির সংখ্যা i এর সর্বনিম্ন no-zero number হয়।

(স্লাইড সময় : 05:25 দেখুন)

সুতরাং আসুন একটি উদাহরণের মধ্যে এটি দেখি। সুতরাং আসুন আমরা অনুমান করি যে আমাদের পাঁচটি প্রসেস p 1 থেকে p 5 আছে এবং এটি হল num মান। সুতরাং, num array তে পাঁচটি element আছে, এবং এই element গুলির প্রতিটি একটি প্রসেসের অনুরূপ। সুতরাং num 0 হল p1 প্রক্রিয়া, এবং num1 হল p2 এর সাথে সম্পর্কিত, num2 হল p3 এর অনুরূপ ইত্যাদি। এখন আমাদের এই অনুমান করি যে এই সমস্ত প্রক্রিয়াগুলি প্রায়শই একসাথে lock i function চালু করে, মূলত এই সমস্ত প্রসেসগুলি প্রায় একযোগে critical section এ প্রবেশ করতে চায়। তাহলে কি হবে?

(স্লাইড সময় : 06:11 দেখুন)

আসুন আমরা বলি প্রক্রিয়া P 3 চালানো শুরু হয়। সুতরাং, এটা এখানে আসে এবং এটি খুঁজে পায়, CPU খুঁজে পায় যায় যে উপস্থিত সমস্ত সংখ্যার যা MAX হয় 0, তাই P 3 এর অনুরূপ একটি num 1 এ সেট করা হয়।

(স্লাইড সময় 06:29 দেখুন)

তারপর আমাদের বলুন P 4 execute হয় এবং এটির num এর ভ্যালু 4 equal to 2 হয়।

(স্লাইড সময় 06:39 দেখুন)

তারপর P 5 execute হয় এবং P 2 execute হয় এবং তারা 3 এবং 4 হিসাবে num এর সংশ্লিষ্ট মান পায়। এখন, আসুন আমরা বলি যে আমরা লুপের এই অংশে এসেছি, এবং আমরা দেখতে পাচ্ছি যে num এর ন্যূনতম non-zero মান সহ প্রক্রিয়াটি critical section এ প্রবেশ করবে। সুতরাং এই বিশেষ ক্ষেত্রে, আমরা num এর এই সমস্ত বিশেষ মানগুলির মাধ্যমে স্ক্যান করি এবং আমরা দেখি যে P 3 এর সর্বনিম্ন মান রয়েছে। অতএব, P 3 কে critical section এ চালানো প্রয়োজন।

(স্লাইড সময় 07:10 দেখুন)

সুতরাং, p3 টি critical section এ চালানো হয় এবং critical section এর শেষে এটি সংশ্লিষ্ট মানকে 0 তে সেট করে দেয়, কারণ অন্যান্য প্রসেসগুলি এই বিশেষ লুপে অপেক্ষা করছে।

(স্লাইড সময় 07:25 দেখুন)

সুতরাং, পরবর্তী সর্বনিম্ন সংখ্যা যা প্রক্রিয়া p4 এর মান 2 যা critical section এ execute হয়। অতএব, প্রক্রিয়া p4 critical section এ প্রবেশ করে এবং শেষে প্রক্রিয়া p4 0 তে সেট করা হয়। এবং তারপর p5 execute করে এবং তারপর p2 execute করে।

(স্লাইড সময় 07:44 দেখুন)

সুতরাং, প্রসেস P 2 চালানো হবে, কারণ এটি শুধুমাত্র nonzero সংখ্যা যা উপস্থিত। সুতরাং, P 2 execution শেষে, আমরা num এর সমস্ত মান 0 পাব।

(স্লাইড সময় 08:06 দেখুন)

সুতরাং, আমাদের এখানে গঠিত একটি প্রয়োজন বা একটি ধারণা হল এই যে এই বিশেষ কাজ MAX atomic হতে হবে; মূলত এটি নিশ্চিত করতে প্রয়োজন যে কোনও দুটি প্রক্রিয়া একই token ঠিক করে না।

(স্লাইড সময় 08:23 দেখুন)

মূলত, এর মানে হল যে যখন একটি নির্দিষ্ট প্রক্রিয়ায় এই নির্দিষ্ট স্টেটমেন্টটি কার্যকর করা হয় যা এই সমস্ত সংখ্যাগুলির MAX খুঁজে পায় এবং 1 যোগ করে তখন কোন context switch প্রদর্শিত হতে পারে না। এই পুরো বিবৃতি একটিমাত্র entity হিসাবে সংগঠিত হয়। সুতরাং, কেন এই বিশেষ ধারণা অনুমান করা হয় যে আমরা কোন দুটি প্রসেস একই সংখ্যা পেতে না তা নিশ্চিত করা প্রয়োজন। তাই আসুন আমরা দেখি কি হবে যদি প্রকৃতপক্ষে একই সংখ্যায় দুটি প্রসেস থাকে, তাহলে মূলত কি হবে যদি এই প্রবেশপথ অথবা এই বিবৃতিটি দ্বারা atomic না হয়। সুতরাং, আমরা আমাদের পাঁচটি প্রসেসের উদাহরণ গ্রহণ করব এবং আমরা এই বিশেষ উদাহরণের সাপেক্ষে দেখাবো।

(স্লাইড সময় 09:19 দেখুন)

সুতরাং, স্বাভাবিক হিসাবে আমাদের প্রক্রিয়া P 3 টি লককে প্রথমে আহ্বান করে, এবং এটি একটি সংখ্যা 1 গ্রহণ করে কারণ, অন্য সমস্ত সংখ্যার সর্বনিম্ন সংখ্যা হল 0।

(স্লাইড সময় 09:34 দেখুন)

তারপর আসুন আমরা অনুমান করি যে প্রক্রিয়া P 4 এবং P 5 একযোগে MAX চালনা করে, যার ফলে তাদের উভয়ের মান 2 পাওয়া যায়।

(স্লাইড সময় 09:45 দেখুন)

এবং অবশ্যই, আমাদের প্রসেস p2 এর মানটি পাওয়া যায় 3। এখন, এই লকের দ্বিতীয় অংশ কি হবে? সুতরাং, আমাদের স্বাভাবিক প্রক্রিয়া হিসাবে দেখতে হবে p3 প্রথম চালানো হচ্ছে, কারণ এটির সর্বনিম্ন সংখ্যা আছে। এবং একবার যখন critical section থেকে বেরিয়ে যায়, p3 তার সংশ্লিষ্ট সংখ্যাটিকে 0 তে সেট করে, তাই p3 এর সাথে সংশ্লিষ্ট এই নম্বরটি 0 তে নির্ধারিত হয়। পরবর্তীতে P4 এবং P 5 এর সাথে সংশ্লিষ্ট দুটি ছোট সংখ্যা সমান হয়। সুতরাং, এর ফলে, P4 এবং সেইসাথে প্রক্রিয়া P5 যা critical section এ একযোগে প্রবেশ করে। এবং এইভাবে আমরা mutual exclusion অর্জন করি নি, তাই এই MAX অপারেশন atomic হওয়ার প্রয়োজন। সুতরাং, এটি নিশ্চিত করবে যে দুটি প্রসেসের num এর জন্য একই মান পাবে না; এবং এর দ্বারা এটি নিশ্চিত করবে যে কোনও নির্দিষ্ট সময়কালের মধ্যে শুধুমাত্র একটি প্রক্রিয়া দ্বারা critical section কার্যকর করা হয়। পরবর্তী দেখব, মূল বেকারি অ্যালগোরিদম যা এই বিবৃতি atomic করতে হবে না।

(স্লাইড সময় 11:05 দেখুন)

মূল বেকারি অ্যালগরিদম নিম্নরূপ; shared array num ছাড়াও একটি shared array যার নাম choosing আছে। সুতরাং, এটি একটি boolean array এবং true এবং false মান থাকতে পারে; এবং এই array এর দৈর্ঘ্য হল N যে প্রতিটি প্রক্রিয়ার একটি নির্দিষ্ট element choosing এ থাকবে। সুতরাং, মূলত এই বিশেষ choosing টি true তে সেট করা হয় প্রক্রিয়া MAX আহ্বান করার আগে এবং এই বিশেষ MAX function invoke করার পরে এবং 1 যোগ করে তারপর প্রক্রিয়া তার choosing false সেট করে। সুতরাং, অ্যালগোরিদম দ্বিতীয় অংশে কিছু ছোটখাট পরিবর্তন আছে। প্রথমত, আমাদের এখানে একটি বিবৃতি while choosing P আছে। সুতরাং, এই বিশেষ বিবৃতিটি নিশ্চিত করবে যে একটি প্রক্রিয়াকে সামনেই নেই যে এটি নিশ্চিত করবে যে বর্তমানে এই MAX এর মাধ্যমে একটি নতুন সংখ্যা স্থাপন করা হচ্ছে না, এটি প্রক্রিয়াটি num এর একটি নতুন মান নির্বাচন করে না।

(স্লাইড সময় 12:26 দেখুন)

দ্বিতীয়ত, যখন লুপের দ্বিতীয় অংশ রয়েছে যা মূলত condition check এ পরিবর্তিত হয়, একটি num of p, $p < \text{num of } i$, i এর একটি tuple আছে। এর মানে যদি $a, b < c, d$ হয় বা $a < c$ or $a == c$ and $b < d$ সমান হয়। দুইটি প্রসেসের একই num হলে এই বিশেষ জটিল চেক শর্ত ভাঙ্গার জন্য ব্যবহার করা হয়। সুতরাং, যদি আমরা দুইটি প্রসেস সংখ্যা num এর একই মান দেওয়া হয় আগে দেখা যায়, এই শর্তটি সমস্যা সমাধানের জন্য ব্যবহার করা হয়, এবং এই দুই প্রসেসের শুধুমাত্র একটি critical section প্রবেশ করবে তা নিশ্চিত করা। যখন num of p আর num of i সমান হয়, অর্থাৎ উভয় সংখ্যারই একই মান থাকে আমাদের একটি প্রসেসকে গুরুত্ব দেওয়া প্রয়োজন। সুতরাং এই ক্ষেত্রে, আমরা i এর ছোট মান দিয়ে প্রক্রিয়া অনুমোদন করব।

(স্লাইড সময় 13:44 দেখুন)

সুতরাং আসুন আমরা একই উদাহরণের সাথে এটি দেখি যেটা আমরা আগে দেখেছি। তাই আসুন আমরা এই উদাহরণটি আবার দেখি এবং আমাদের স্বাভাবিক প্রক্রিয়া হিসাবে বলি P3 প্রথমে এই MAX execute করে এবং সর্বনিম্ন সংখ্যা দেওয়া হয়, তারপর P4 এবং P5 টি একই সংখ্যা 2 পেতে এবং তারপর P2 প্রক্রিয়াটি পরবর্তী সর্বোচ্চ মান 3 পায়। এখন, আমাদের এই লকিং এর দ্বিতীয় অংশে তাকান যাক। সুতরাং, critical section এ চালানোর প্রথম প্রক্রিয়া হল P3, কারণ এটির সর্বনিম্ন সংখ্যা আছে, সুতরাং P3 execute করে। এবং শেষ পর্যন্ত, এটি num কে 3 থেকে 0 করে। এখন পরবর্তী প্রক্রিয়াকরণটি P4 বা P5 হতে পারে। তাই, আমরা এই দুটি প্রক্রিয়ার মধ্যে কিভাবে নির্বাচন করব? আমরা দেখেছি যে num of P এবং আর num of i উভয় সংখ্যা এইরকম দুটি ক্ষেত্রে আছে, তাই, এই পদ্ধতিতে একটিকে সমর্থন করার জন্য আমরা দ্বিতীয় অংশটি দেখি যা P এবং i, এটির উপর ভিত্তি করে এটির পক্ষে আমরা সমর্থন করি সেই প্রক্রিয়া যা একটি নিম্ন সংখ্যা এবং অতএব, প্রক্রিয়া P4 critical section এ সঞ্চালিত হয়। সুতরাং, P4 চালানোর পরে তার মান 0 সেট করা হয়, এবং তারপর বেশ স্বাভাবিকভাবেই P5 execute করে। এবং P 5 এর পরে স্বাভাবিক হিসাবে P 2 চালানো হবে। এইভাবে আমরা boolean array টি নির্বাচন করার পাশাপাশি আরও জটিল শর্তসাপেক্ষ চেকটি MAX- এর একটি atomic operation প্রয়োজনীয়তা সমাধান করতে সহায়তা করে। সুতরাং, এটিই মূল বেকারি অ্যালগরিদম যা লেসলি ল্যামপোর্ট কর্তৃক প্রস্তাবিত; এবং এটি কার্যকরীভাবে critical section সমস্যার সমাধান করতে সাহায্য করে যখন প্রসেসের সংখ্যা 2 এর থেকে বেশি। ধন্যবাদ।